

Quality Attributes

Lecturer: Ngo Huy Bien
Software Engineering Department
Faculty of Information Technology
VNUHCM - University of Science
Ho Chi Minh City, Vietnam
nhbien@fit.hcmus.edu.vn

Objectives

- To *elicit* and *document* architectural requirements for software system.
- To *create* a simple architectural design to ensure scalability.



Contents

- Quality goals.
- Quality attributes.
- Quality attribute scenarios.
- Tactics.



References

1. Len Bass, Paul Clements and Rick Kazman (2021). Software Architecture in Practice. Addison-Wesley.
2. R. Kazman, M. Klein and P. Clements (1998). Evaluating Software Architectures for Real-Time Systems.
3. Rick Kazman, Mark Klein and Paul Clements (2000). ATAM: Method for Architecture Evaluation.

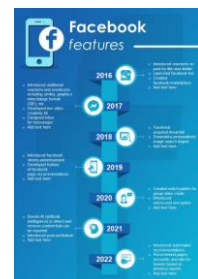


Functionality [1]

- *Functionality* is the ability of the system to do the work for which it was intended.
- If functionality were *the only thing* that mattered, you wouldn't have to divide the system into pieces at all; a single monolithic blob with no internal structure would do just fine.
- But although functionality is independent of any particular structure, functionality is achieved by *assigning responsibilities to architectural elements*, resulting in one of the most basic of architectural structures.
- The architect's interest in functionality is in how it *interacts with* and *constrains* other qualities.

Example: Facebook Functionalities

- Registration
- Login
- Profile
- Posts
- Timeline
- Friends
- Likes
- Comments
- Images
- Videos



Quality Requirements

- Scalability
- Availability
- Performance
- Security
- Modifiability
- Extensibility



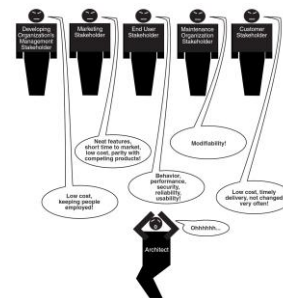
If a complex system fails to meet the quality requirements, it is useless.

Exercise: Scalability

- 200,000 requests in 5 minutes.
- How to test scalability?
 - JMeter
 - Pingdom
- How to achieve scalability?
 - Load balancing (nginx, HAProxy)
 - Caching (Redis, Memcached)
- **Exercise:** Create an architectural diagram to ensure scalability.
 - Browsers, load balancer, web servers, database server, cache servers.



Stakeholders' Goals



Quality Goals [2]

- **Quality goals** form the basis for architectural evaluation, but quality goals by themselves are not a sufficient basis to judge an architecture for suitability.
 - The system shall be **robust**.
 - The system shall be **highly modifiable**.
 - The system shall be **secure** from unauthorized break-in.
 - The system shall exhibit acceptable **performance**.



Rabbit or Duck

Measuring Quality Attribute [3]

- Evaluating an architectural design against quality attribute requirements necessitates a **precise characterization** of the quality attributes of concern.
- For example, understanding an architecture from the point of view of **modifiability** requires an understanding of **how to measure or observe modifiability** and an understanding of how various types of architectural decisions impact this measure.



Quality Attribute Characterizations

- *Quality attribute characterizations* answer the following questions about each attribute:
 - What are the *stimuli* to which the architecture must respond?
 - *External stimuli* (or just stimuli for short) are the events that cause the architecture to respond or change.
 - What is the *measurable or observable* manifestation of the quality attribute by which its achievement is judged?
 - What are the *key architectural decisions* that *impact achieving* the attribute requirement?
 - *Architectural decisions* are those aspects of an architecture — components, connectors, and their properties—that have a direct impact on achieving attribute responses.

Example Quality Attribute: Performance

- For example, the external *stimuli for performance* are events such as messages, interrupts, or user keystrokes that result in computation being initiated.
- *Performance architectural decisions* include processor and network arbitration mechanisms; concurrency structures including processes, threads, and processors; and properties including process priorities and execution times.
- *Responses* are characterized by measurable quantities such as latency and throughput.



Example Quality Attribute: Modifiability

- For *modifiability*, the external *stimuli* are change requests to the system's software.
- *Architectural decisions* include encapsulation and indirection mechanisms, and
- the *response* is measured in terms of the number of affected components, connectors, and interfaces and the amount of effort involved in changing these affected elements.



Why Use Attribute Characterizations?

- The attribute characterizations help to ensure attribute coverage as well as offering a rationale *for asking elicitation questions*. (REQUIREMENTS ELICITATION)
- Examples of *performance related questions*:
 - What is the physical location of the hardware and its connectivity?
 - What are the bandwidth characteristics of the network?
 - Do you use a synchronous or an asynchronous protocol?
 - What is the location of firewalls and their impact on performance?
 - What information is cached versus regenerated? Based upon what principles?
 - How are resources allocated to service requests?
 - How do we characterize client loading, (e.g., how many concurrent sessions, how many users)?
 - What are the performance characteristics of the middleware: load balancing, monitoring, reconfiguring services to resources?



Scenarios

- Typically the first job of an architecture analysis is to precisely *elicit the specific quality goals* against which the architecture will be judged. The *mechanism* that we use for this elicitation is the scenario.
- A *scenario* is a short statement describing an interaction of one of the stakeholders with the system.
 - A *user* would describe using the system to perform some task; his scenarios would very much resemble use cases in object-oriented parlance.
 - A *maintainer* would describe making a change to the system, such as upgrading the operating system in a particular way or adding a specific new function.
 - A *developer's* scenario might talk about using the architecture to build the system or predict its performance.
 - A *customer's* scenario might describe how the architecture is to be re-used for a second product in a product line.

Use Case Scenarios

- *Use case scenarios* describe a user's intended interaction with the completed, running system.
 - The user wants to examine budgetary and actual data under different fiscal years without re-entering project data. (**usability**)
 - User changes graph layout from horizontal to vertical and graph is redrawn in one second. (**performance**)

Growth Scenarios

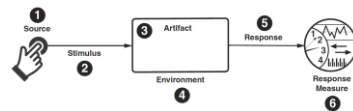
- *Growth scenarios* represent typical anticipated future changes to a system.
 - The maximum number of tracks to be handled by the system doubles and the maximum latency of track data to the screen is kept to **200 ms**.
 - Migrate to a new operating system, or a new release of the existing operating system in less than a **person-year of work**.

Exploratory Scenarios

- *Exploratory scenarios* push the envelope and stress the system.
 - Improve the system's **availability** from 98% to 99.999%.
 - Half of the servers go down during normal operation without affecting overall system **availability**.

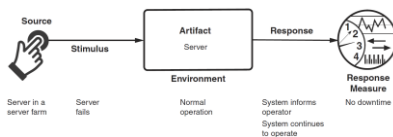
Quality Attribute Scenario Representation [1]

- Quality attribute scenarios have six parts: Stimulus, Source, Response, Response Measure, Environment, Artifact.
- The environment is the set of circumstances in which the scenario takes place.

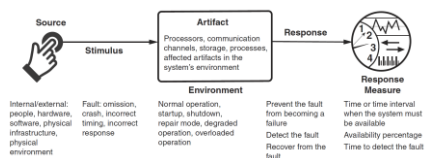


Example: Availability Scenario

- *Availability* refers to a property of software that it is there and ready to carry out its task when you need it to be.
- Mean Time Between Failure (MTBF), Mean Time To Recovery (MTTR).

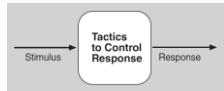


More Complex Availability Scenario



Architectural Tactics

- **Architectural tactics** – the techniques an architect can use to achieve the required quality attributes.
- A **tactic** is a design decision that influences the achievement of a quality attribute response—tactics directly affect the system's response to some stimulus.

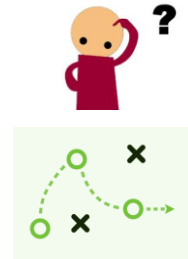


- If **no pattern exists** to realize the architect's design goal, **tactics** allow the architect to construct a design fragment from "first principles."

Example: Tactics for Availability

- How to design an architecture for availability?

- Detect Faults.
 - Ping/echo.
 - Monitor.
 - Heartbeat.
 - Timeout.
- Recover from Faults.
 - Active redundancy (hot spare).
 - Passive redundancy (warm spare).
 - Rollback.
 - Retry.
- Prevent Faults.
 - Transactions. (ACID properties.)
 - Exception prevention.



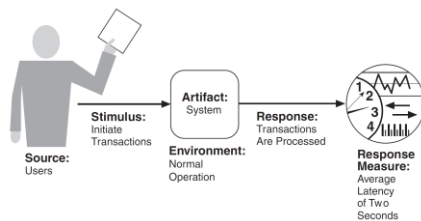
Tools for Availability

- <https://uptimerobot.com/>
- <https://www.pingdom.com/>
- <https://jmeter.apache.org/>



Performance Scenario

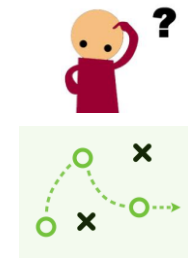
- **Performance**, that is: It's about time and the software system's ability to meet timing requirements.



Tactics for Performance

- How to design an architecture for performance?

- Control Resource Demand.
 - Manage sampling rate.
 - Limit event response.
 - Prioritize events.
 - Reduce overhead.
 - Bound execution times.
 - Increase resource efficiency.
- Manage Resources.
 - Increase resources.
 - Introduce concurrency.
 - Maintain multiple copies of computations.
 - Maintain multiple copies of data.
 - Bound queue sizes.
 - Schedule resources.



Tools for Performance

- <https://developer.android.com/studio/profile>
- <https://developer.apple.com/videos/play/wwdc2019/411/>
- <https://locust.io/>



Security Concepts (I)



- The simplest *approach to characterizing security* has three characteristics: confidentiality, integrity, and availability (CIA):
 - **Confidentiality** is the property that data or services are protected from unauthorized access. For example, a hacker cannot access your income tax returns on a government computer.
 - **Integrity** is the property that data or services are not subject to unauthorized manipulation. For example, your grade has not been changed since your instructor assigned it.
 - **Availability** is the property that the system will be available for legitimate use. For example, a denial-of-service attack won't prevent you from ordering book from an online bookstore.

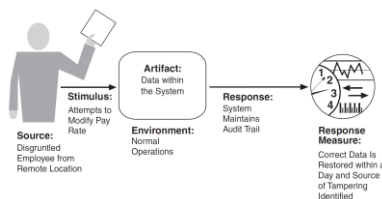
Security Concepts (II)



- **Other characteristics** that are used to support CIA are these:
 - **Authentication** verifies the identities of the parties to a transaction and checks if they are truly who they claim to be. For example, when you get an email purporting to come from a bank, authentication guarantees that it actually comes from the bank.
 - **Nonrepudiation** guarantees that the sender of a message cannot later deny having sent the message, and that the recipient cannot deny having received the message. For example, you cannot deny ordering something from the Internet, or the merchant cannot disclaim getting your order.
 - **Authorization** grants a user the privileges to perform a task. For example, an online banking system authorizes a legitimate user to access his account.

Security Scenario

- **Security** is a measure of the system's ability to protect data and information from unauthorized access while still providing access to people and systems that are authorized.



Tactics for Security

- How to design an architecture for security?

- Detect Attacks
 - Detect service denial.
 - Verify message integrity.
- Resist Attacks
 - Authenticate actors.
 - Authorize actors.
 - Limit access.
 - Limit exposure.
 - Encrypt data.
 - Separate entities.
- React to Attacks
 - Revoke access.
 - Lock computer.
 - Inform actors.



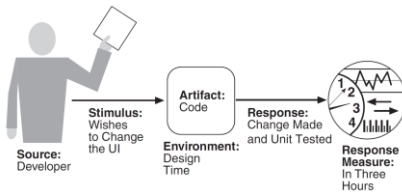
Tools for Security

- <https://www.acunetix.com/>
- https://owasp.org/www-community/Vulnerability_Scanning_Tools



Modifiability Scenario

- **Modifiability** is about change, and our interest in it centers on the cost and risk of making changes.



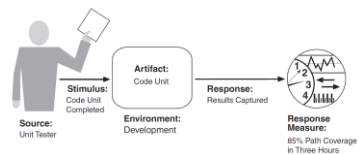
Tactics for Modifiability

- How to design an architecture for modifiability?
 - Reduce Size of a Module.
 - Increase Cohesion.
 - Reduce Coupling.
 - Defer Binding.



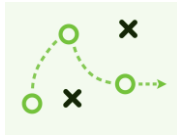
Testability Scenario

- Software **testability** refers to the ease with which software can be made to demonstrate its faults through (typically execution-based) testing.
- Intuitively, a system is testable if it “gives up” its faults easily. If a fault is present in a system, then we want it to **fail during testing as quickly as possible**.



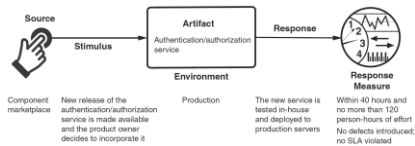
Tactics for Testability

- How to design an architecture for testability?
 - Control and Observe System State
 - Specialized interfaces
 - Abstract data sources
 - Limit Complexity
 - Limit structural complexity
 - Limit nondeterminism

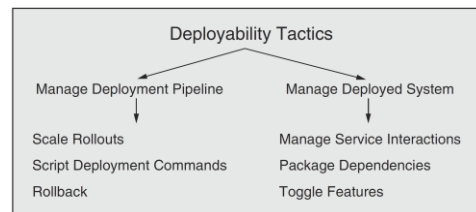


Deployability Scenario

- Deployability refers to a property of software indicating that it may be deployed—that is, allocated to an environment for execution—within a predictable and acceptable amount of time and effort.

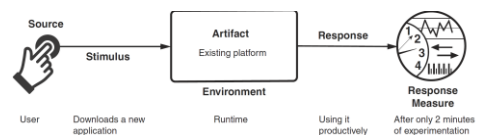


Tactics for Deployability

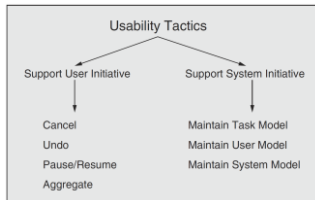


Usability

- Usability** is concerned with how easy it is for the user to accomplish a desired task and the kind of user support that the system provides.



Tactics for Usability



Maintain Task Model

- The **task model** is used to determine context so the system can have some idea of what the user is attempting to do and provide assistance.
- For example, many search engines provide **predictive type-ahead capabilities**, and many mail clients provide spell-correction. Both of these functions are based on task models.



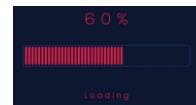
Maintain User Model

- The **user model** explicitly represents the user's knowledge of the system, the user's behavior in terms of expected response time, and other aspects specific to a user or a class of users.
- For example, language-learning apps are constantly monitoring areas **where a user makes mistakes** and then providing additional exercises to correct those behaviors.
- A special case of this tactic is commonly found in user interface customization, wherein a user can explicitly modify the system's user model.



Maintain System Model

- The **system** maintains an explicit model of itself. This is used to determine expected system behavior so that appropriate feedback can be given to the user.
- A common manifestation of a system model is **a progress bar** that predicts the time needed to complete the current activity.



Other Kinds of Quality Attributes

System/Software Product Quality								
Functional Initiability	Performance Efficiency	Compatibility	Usability	Reliability	Security	Maintainability	Portability	
Functional Completeness	Time behavior	Co-existence	Appropriateness/compatibility	Maturity	Confidentiality	Modularity	Portability	
Functional Correctness	Resource utilization	Interoperability	Learnability	Availability	Integrity	Reusability	Adaptability	
Functional Appropriateness			Operability	Fault tolerance	Non-repudiation	Analysability	Immutability	
			User error protection	Recoverability	Accountability	Changeability		
			User interface modifiability		Authenticity	Testability		
			Accessibility			Modification variability		

ISO/IEC FCD 25010: Systems and Software Engineering: Systems and Software Product Quality Requirements and Evaluation (SQuaRE): System and Software Quality Models.



Thank You & See You Again

